

04 — File-by-file reference

Every file in the repo, grouped by folder, with its purpose and the key things inside it. Line counts are approximate and just give a sense of size. Generated artifacts, node_modules, and the dataset are omitted.

Entry points (repo root)

File	~Lines	Purpose
precompute.py	322	Phase 1 (offline). Builds every artifact rank.py reads: features, embeddings, BM25 index, the trained model, rerank + tournament scores. Has the content-hash cache and --resume, --no-st, --no-accuracy, --limit, --bm25-backend flags.
rank.py	282	Phase 2 (online). No network, CPU, < 5 min. Loads artifacts, predicts fit, merges scores, drops honeypots, takes top 100, writes reasons, validates. Has a fast path (cached features) and a live path (compute features on the fly).
validate_submission.py	166	The organiser's official validator — checks the CSV against the challenge rules. Treated as read-only; we run it but don't edit it.

core/ — cross-cutting foundations

File	~Lines	Purpose
config.py	299	Every tunable in one place , read from environment / .env. Uses pydantic-settings if available, else a plain-dataclass fallback. Holds weights, paths, model names, device/ONNX flags, training knobs, DB/Redis/Neo4j URLs, JWT settings, etc.
constants.py	373	Single source of truth for domain vocabularies (skill taxonomies: retrieval, LLM, core-ML, CV/speech/robotics; consulting firms) and NUMERIC_FEATURE_COLUMNS — the 40-feature ML contract.
logging.py	151	Structured JSON logging to stdout via structlog, plus a log_duration context manager that times a block and attaches metrics.
security.py	116	Passwords (bcrypt), JWTs (PyJWT), and API keys. Hash/verify passwords, mint/decode tokens, generate+hash API keys.
device.py	96	Performance. Picks the best compute device (MPS/CUDA/CPU) and builds the optimised→plain constructor attempts for the transformers (ONNX-int8 → ONNX → torch). Never raises.

File	~Lines	Purpose
llm.py	175	One LLM call for the whole project. <code>complete()</code> / <code>complete_json()</code> try GLM (Z.ai, OpenAI-compatible) → local Ollama → Gemini → None (callers use heuristics). Used by JD parse, summaries, label grading, NL query. Performance. Content-addressed cache: each stage stores a <code><file>.key</code> sidecar; a re-run reuses the artifact only if the recomputed key matches. A stale/unreadable cache is a clean miss, never a wrong result. A small async circuit breaker — opens after repeated failures (e.g. GitHub down), short-circuits calls, half-opens to test recovery. Domain exceptions, each carrying a stable machine code and an HTTP status, so the API can translate them into clean error responses. OpenTelemetry setup (optional). Configures spans/exporter to Jaeger when enabled; a no-op otherwise. The Celery app for ingestion/intelligence workers + the beat scheduler. Defaults to “eager” (runs inline) so no broker is needed in dev.
artifact_cache.py	89	
circuit_breaker.py	75	
exceptions.py	95	
tracing.py	91	
celery_app.py	53	

pipeline/ — the ranking engine

File	~Lines	Purpose
loader.py	136	Streaming loader. <code>iter_candidates()</code> (one profile at a time), <code>corpus_text()</code> (full searchable doc), <code>rerank_text()</code> (focused doc for the cross-encoder), <code>file_sha256()</code>, <code>count_lines()</code>. The 40-feature builder. <code>build_feature_row()</code> + <code>frame_from_rows()</code>. Also <code>apply_live_signals()</code> (adds the evidence-bonus columns, default 0) and skill-confidence helpers. The biggest, most central pipeline file. High-precision rule-based detection of the ~80 fake profiles (impossible career arithmetic). Tuned to avoid false positives. JD → JobSpec. The only place an LLM (Gemini) may run, offline only; rich rule-based fallback. Saves <code>jd_spec.json</code>. Hard gates at ranking time (vectorised): consulting-only careers, keyword stuffers, inactive/unresponsive. Writes the rejection registry.
feature_engineering.py	486	
honeypot_detector.py	106	
jd_deconstructor.py	297	
gate_filter.py	91	

File	~Lines	Purpose
embedder.py	157	Semantic embeddings. SentenceTransformerEmbedder (with device/ONNX) and the NumPy HashingEmbedder fallback; embed_corpus(), make_embedder(), meta save/load for offline rebuild.
bm25_indexer.py	144	BM25 keyword index. rank-bm25 backend + a memory-frugal streaming inverted-index fallback with DF pruning. build_index(), query_scores(), save/load.
retrieval.py	68	Hybrid retrieval. cosine_scores() (dense) + fuse_rrf() (Reciprocal Rank Fusion of BM25 and embeddings).
scorer.py	91	Final-score merge — the one formula, shared by precompute and rank. Convex head-blend of rerank/tournament, location & signal multipliers, evidence bonus, honeypot/gate pinning.
signal_scorer.py	46	Behavioural sub-score (availability, engagement, trust, GitHub) and the signal_modifier down-weight, vectorised over the matrix.
tournament.py	126	Our algorithm. Pairwise round-robin over the head, resolved by a Bradley-Terry MLE (Hunter's MM iteration). Pure NumPy. run_tournament().
loop_agent.py	128	Coverage loop (plan→retrieve→reflect), pure Python. Reports shortlist coverage; informational, doesn't reorder.

ml/ — models, labels, metrics

File	~Lines	Purpose
trainer.py	304	Training. create_proxy_labels(), train_fit_scorer() (regressor, Strategy pattern over XGBoost/LightGBM/sklearn/formula), and train_ranker() (LambdaMART on graded labels) with the downsample + histogram + early-stopping speed levers.
predictor.py	53	Prediction. predict_fit() loads the model artifact and returns normalised fit scores; falls back to the proxy formula if the model is missing/unloadable.
pseudo_labels.py	129	Graded 0-5 labels for learning-to-rank. Percentile tiering (_bucket_to_tiers), optional Gemini grading of a sample, heuristic otherwise.
reranker.py	152	Cross-encoder reranker of the head (sentence-transformers CrossEncoder), with a pure-NumPy lexical fallback. Device/ONNX aware.

File	~Lines	Purpose
llm_scorer.py	80	LLM head-scoring — <code>score_head()</code> has the LLM (GLM/Ollama/Gemini) read the top ~250 vs the JD and score fit 0-100, batched + cached as <code>llm_fit_score</code> . Offline; opt-in.
evaluate.py	78	Eval harness — <code>ndcg_at_k</code> , <code>average_precision</code> , <code>precision_at_k</code> , the weighted composite, plus <code>compare()</code> and <code>lift()</code> for A/B.

output/ — results

File	~Lines	Purpose
reasoning_generator.py	112	Turns a feature row into a varied, JD-relevant one-line justification (template-based, no LLM at rank time).
submission_writer.py	87	Assembles exactly 100 rows, writes <code>submission.csv</code> , and runs the official validator on it.

api/ — the FastAPI service

File	~Lines	Purpose
app.py	144	App factory — lifespan (startup/shutdown), middleware, error handlers, router registration, metrics/tracing wiring.
main.py	25	Uvicorn entry point: <code>uvicorn api.main:app</code> .
deps.py	113	Dependencies — singletons from <code>app.state</code> , auth (API key or JWT), RBAC role checks, DB session.
schemas.py	214	Pydantic request/response models — validation at the edge.
middleware.py	52	Per-request request-id , structured access logging, metrics timing.
metrics.py	42	Prometheus counters/histograms; the <code>/metrics</code> exposition.
errors.py	48	Exception handlers → clean JSON error bodies.
routers/auth.py	119	Login → JWTs; create/list/revoke API keys (shown once).
routers/rank.py	152	Ranking endpoints — submit a job, poll status, stream progress (SSE), get results.
routers/developers.py	132	Live layer — connect a GitHub account, watch the sync, read the proof-of-work profile.
routers/search.py	43	Natural-language search over the evidence graph.
routers/admin.py	85	Artifact status, trigger precompute, inspect audit logs/jobs.
routers/health.py	63	Liveness (process up) + readiness (artifacts/DB ready).

db/ — persistence

File	~Lines	Purpose
models.py	212	ORM models (users, API keys, jobs, results, audit, developers, evidence). UUID string PKs + portable JSON columns so the same schema runs on Postgres and SQLite.
repositories.py	348	Repository layer — all DB access. Routers/services never write raw SQL.
base.py	110	Async engine + session factory; SQLite WAL/pragma tuning; session_scope().
init.sql alembic/env.py, alembic/versions/*	— 57 + 135	Initial SQL for the Postgres container. Alembic migration environment + the initial schema migration.

services/ — engine wrappers & infra

File	~Lines	Purpose
ranker.py	284	RankerService — loads the engine once and keeps it warm; the rank() method the API/worker call.
jobs.py	246	JobManager — submit (commit-then-enqueue), the idempotent worker handler, persistence, SSE progress channels.
queue.py	247	Message queue — in-memory PriorityQueue + QueueWorker, and a RedisStreamQueue (consumer groups, dead-letter) with the same interface.
cache.py	148	Cache abstraction — Redis backend + in-memory fallback; eviction (LRU/LFU) and write strategies.
ratelimit.py	40	Fixed-window rate limiter (Redis INCR or in-memory).

ingestion/ — the live GitHub layer

File	~Lines	Purpose
github_client.py	241	Async GitHub GraphQL client — fetch repos/commits/PRs/issues within rate limits, guarded by the circuit breaker.
sync.py	141	Ingestion core — shared by the API (await) and Celery (background); turns raw activity into evidence and writes the graph.
tasks.py	148	Celery task definitions for ingestion + intelligence; eager fallback.
resume_parser.py	54	Extract text + canonical skills from a resume (PDF/docx); optional.

intelligence/ — evidence reasoning

File	~Lines	Purpose
confidence_scorer.py	90	Confidence formula — source-weighted, complexity-scaled, recency-decayed trust per skill claim. Map free-text/messy skill strings to canonical names (spaCy optional, taxonomy fallback). Turn a raw artifact (commit diff, PR review, issue, SO answer) into a short summary (Gemini optional, heuristic fallback).
skill_extractor.py	107	
summariser.py	171	

graph/ — the proof-of-work knowledge graph

File	~Lines	Purpose
schema.py	176	Graph schema + an in-memory backend (the fallback when Neo4j is off); <code>all_evidence()</code> feeds the vector index.
queries.py	230	GraphWriter / GraphQuerier over a pluggable backend (Neo4j or in-memory); confidence accumulates across evidence.
vector_index.py	130	Semantic index over evidence summaries (embeddings + cosine) for hybrid retrieval; reuses <code>make_embedder</code> , cached per evidence count.
natural_language_query.py	165	Recruiter English → hybrid retrieval: structured skill traversal + semantic vector search, fused with RRF; results tagged <code>match_via</code> .
evaluate.py	90	Graph-RAG eval harness — <code>precision@k</code> , <code>recall@k</code> , <code>MRR</code> , <code>hit@k</code> , <code>compare()</code> + <code>lift()</code> against a gold set.

frontend/ — React recruiter UI

File	Purpose
src/main.tsx, src/App.tsx	App bootstrap + routes.
src/pages/Login.tsx	Login screen (JWT auth).
src/pages/Dashboard.tsx	Run a ranking, see job status.
src/pages/RankingResults.tsx	The ranked shortlist + reasons.
src/pages/NaturalLanguageSearch.tsx	Query the evidence graph in English.
src/pages/Settings.tsx	API keys, account settings.
src/components/Layout.tsx, components/ui.tsx	Shell + reusable UI primitives.
src/components/developer/EvidenceTimeline.tsx, ProofOfWorkCard.tsx	Live-layer visualisations.
src/hooks/useJobStream.ts	Subscribes to the SSE job-progress stream.
src/lib/api.ts, src/lib/auth.tsx	API client + auth context.
vite.config.ts, tsconfig*.json, package.json, Dockerfile	Build/config.

Infrastructure & config

File	Purpose
docker-compose.yml	13 services — postgres, redis, precompute, api2 (replica), neo4j, worker-ingestion, worker-intelligence, beat, gateway (nginx), frontend, prometheus, grafana, jaeger + volumes. Multi-stage images for the API and the UI. CI: install, lint (Ruff), run the test suite. Prometheus scrape config + Grafana dashboards/datasource provisioning. All Python dependencies (every heavy one is optional at runtime). Tooling config (Ruff, pytest, Alembic). Build a small stratified sample dataset for fast local runs. Hackathon submission metadata.
Dockerfile, frontend/Dockerfile	
.github/workflows/ci.yml	
deploy/prometheus/*, deploy/grafana/*	
requirements.txt	
pyproject.toml, pytest.ini, alembic.ini	Tooling config (Ruff, pytest, Alembic). Build a small stratified sample dataset for fast local runs. Hackathon submission metadata.
scripts/make_dev_sample.py	
submission_metadata*.yaml	

tests/ — the safety net (68 tests)

File	Covers
conftest.py	Shared fixtures + a candidate factory that emits schema-valid profiles (and specific edge cases: honeypots, stuffers).
test_feature_engineering.py	The 40-feature builder.
test_honeypot_detector.py	Honeypot rules (precision).
test_gate_filter.py	Hard gates.
test_accuracy_layer.py	Eval metrics, graded pseudo-labels, reranker, tournament, scorer head-blend.
test_performance.py	Device selection, focused rerank text, training downsample, the content cache.
test_reasoning_generator.py,	Output + CSV validity.
test_submission_format.py	End-to-end: the live path yields a valid submission, fast. API integration against a temp SQLite DB + in-memory cache.
test_rank_smoke.py	
test_api.py	Queue/worker + cache eviction.
test_queue.py	Confidence formula, summariser, skill extraction, graph.
test_live_layer.py	Hybrid graph+vector retrieval vs skill-only on a gold set (recall/MRR lift), eval metrics, vector index.
test_graph_rag.py	

Next: [05_build_from_scratch.md](#) to put it all together yourself.